

Why Math Beats AI/ML: Evidence Based Detection

Executive Summary

AI/ML detects what it has seen before. Math detects what is happening now.

The Zantoras Evidence Engine introduces a new category of cybersecurity: **Evidence Based Detection**. Instead of training models on historical attacks and hoping they generalize, the Evidence Engine deploys 26 pure-mathematical detectors that measure the physical and statistical properties of data in real time. The output is not a probability score -- it is a deterministic measurement, reproducible by any third party, chained into a tamper-evident cryptographic audit trail.

By the numbers:

- 26 mathematical detectors across 5 independent families
- 223 detection tiles across 16 security categories
- 635 MITRE ATT&CK technique IDs covered across all 14 tactics
- 6 Beyond MITRE exclusive detections no other platform offers
- Zero AI/ML dependencies, zero training data, zero GPU requirements

The difference: AI/ML is a sniffer dog -- powerful but trained on known scents. It finds what it has learned to find, and every new threat requires new training. The Evidence Engine is a Geiger counter -- it measures a physical property of the data that changes regardless of the specific threat. A Geiger counter does not need to have seen a specific isotope before. It measures radiation. The Evidence Engine does not need to have seen a specific ransomware variant before. It measures the mathematical shape of data -- entropy, compression, cadence, correlation, distribution -- and these properties change whenever data is tampered with, encrypted, exfiltrated, or manipulated.

What you get on day one: No training period. No labeled datasets. No tuning. The Evidence Engine learns baselines from live data using exponential moving averages and online statistical algorithms. First actionable detections arrive within minutes of deployment, not weeks. Every detection is deterministic, reproducible, and independently verifiable -- evidence that stands up in court, satisfies insurance auditors, and meets NIST SP 800-53 CP-9.

AI/ML predicts. Math proves. Welcome to Evidence Based Detection.

Table of Contents

1. Executive Summary
2. CISO Brief: Evidence Based Detection in Three Pages
3. The Problem with AI/ML in Cybersecurity
4. Math is Deterministic. AI/ML is Probabilistic.
5. How Math Detects What AI/ML Cannot (26 Detectors)
6. Why This Matters (Detection Coverage by Domain)
7. Beyond MITRE ATT&CK: EE-Exclusive Detections
8. Why Attackers Cannot Evade Math
9. Distinguishing Attacks from Legitimate Changes
10. The Evidence Gap
11. Composite Detection Pipeline

12. Full MITRE ATT&CK Coverage
13. Summary

CISO Brief: Evidence Based Detection in Three Pages

AI/ML detects what it has seen before. Math detects what is happening now.

The Core Comparison

Property	Mathematical Detection	AI/ML Detection
Output	Deterministic measurement	Probability estimate
Zero-day detection	Yes -- measures shape change	Limited -- generalizes within training data
Explainability	"Entropy increased 4.2 sigma"	"Model confidence: 87%"
Court admissibility	Reproducible, verifiable, Daubert-ready	Black box, routinely challenged
Training data	None -- baselines from live data	Large labeled or unlabeled datasets
Model drift	Baselines adapt continuously	Requires periodic retraining
Adversarial resistance	Must manipulate physics of data	Models fooled by adversarial inputs
Day-one readiness	Works immediately	Requires training/tuning period
Computational cost	Lightweight arithmetic	GPU-intensive inference

Five Detector Superpowers

The 26 detectors are organized into five mathematically independent families. An adversary who defeats one family still faces four others.

- **Statistical** (Z-Score, CUSUM, Mahalanobis, EMA, Hurst, Destination Frequency) -- catches volume anomalies, slow drift, correlated metric deviations, behavioral trend shifts, and rare connections. These detectors answer: *is this observation statistically normal?*
- **Information-Theoretic** (Shannon Entropy, Entropy Rate, Compression Ratio, NCD) -- catches encryption, tunneling, covert channels, and synthetic data. These detectors answer: *has the information content of this data changed?*
- **Structural / Distributional** (Benford's Law, Zipf's Law, Fuzzy Hashing, Cosine Similarity, JSD, Entity Diff) -- catches fabricated data, distribution flattening, behavioral shape shifts, and configuration tampering. These detectors answer: *does this data still have its natural structure?*
- **Temporal / Frequency-Domain** (Cadence, Spectral FFT, Wavelet Decomposition) -- catches C2 beaconing, schedule disruption, and multi-timescale attacks. These detectors answer: *has the rhythm of this system changed?*
- **Behavioral Inversion** (Symmetric Traffic, Fixed Payload, Time-of-Day Inversion, Reader-Writer Inversion, Talker Goes Quiet, Schedule Drift, Ghost Session) -- catches tunneling, block-cipher C2, after-hours compromise, account takeover, BEC email suppression, and credential testing. These detectors answer: *has this entity stopped behaving like itself?*

Why Attackers Cannot Evade Math

The five detector families impose contradictory constraints on attackers:

- **To evade entropy detection**, an attacker must reduce randomness in encrypted output -- but reducing randomness weakens the encryption, making it reversible.
- **To evade compression ratio detection**, an attacker must add redundancy to encrypted data -- but added redundancy triggers the NCD and entropy detectors.
- **To evade cadence detection**, an attacker must mimic the timing rhythm of legitimate processes -- but mimicking timing constrains throughput, slowing the attack to the speed of normal operations.
- **To evade Mahalanobis distance**, an attacker must preserve byte-packet-duration correlations -- but preserving all three correlations while exfiltrating data limits volume to near-zero.
- **To evade Zipf's Law detection**, an attacker must concentrate activity on few targets to mimic natural power-law distributions -- but concentration reduces attack surface coverage.

These constraints are contradictory. Solving one exposes the attacker to another. An attacker who satisfies all five simultaneously is operating at the speed, volume, timing, and distribution of legitimate traffic -- meaning the attack has been reduced to a rate indistinguishable from normal operations.

What This Means for Your Business

- **Day-one protection** -- no training period, no labeled data, no tuning. Detections begin within minutes of deployment.
- **Court-admissible evidence** -- every detection is deterministic, reproducible, and chained into a tamper-evident SHA-256 audit trail. Not a probability score -- a mathematical proof.
- **Insurance defensibility** -- 44% of cyber insurance claims were denied in 2024 due to inability to prove controls were operational. The Evidence Engine produces cryptographic proof, not self-attestation.
- **Complete threat coverage** -- 223 detection tiles, 635 MITRE technique IDs across all 14 tactics, plus 6 Beyond MITRE exclusives. No signatures, no feeds, no models.
- **Zero operational overhead** -- lightweight arithmetic runs at line speed on commodity hardware. No GPUs. No model retraining. No data science team required.

The Category: Evidence Based Detection

The Evidence Engine does not belong to the SIEM category, the EDR category, or the AI/ML threat detection category. It belongs to a new category: **Evidence Based Detection** -- detection that produces mathematical evidence, not statistical guesses. Evidence that is deterministic, reproducible, and independently verifiable. Evidence that stands up in court, satisfies regulators, and gives insurers what they actually need: proof.

Technical Deep-Dive

The following sections provide the complete mathematical and technical detail behind Evidence Based Detection.

The Problem with AI/ML in Cybersecurity

The cybersecurity industry has spent a decade betting on machine learning. Train a model on known threats, feed

it data, and hope it generalizes. This approach has a fundamental flaw: **it only finds what it's been taught to look for.**

CyberSense, the industry's leading ransomware detection engine, maintains a database of 7,500+ ransomware variants built from 120 million+ samples. It uses 200+ content-inspection metrics trained on known attack patterns. This is impressive engineering -- and it is structurally fragile.

When a novel ransomware variant encrypts your backup data using a method that doesn't match any of those 7,500 known patterns, the model is extrapolating beyond its training distribution. It produces a probability score. And that score is only as reliable as the statistical similarity between the new threat and whatever the model was trained on.

Zantoras takes a different approach. We use math.

Math is Deterministic. AI/ML is Probabilistic.

Property	Mathematical Detection	AI/ML Detection
Output	Deterministic measurement	Probability estimate
Zero-day detection	Yes -- measures shape change	Limited -- generalizes only within training distribution
Explainability	"Entropy increased 4.2 standard deviations"	"Model confidence: 87%"
Court admissibility	Reproducible, verifiable	Black box, challenged under Daubert
Training data required	None -- learns baselines from live data	Supervised: large labeled datasets; unsupervised: unlabeled corpora; both require representative data
Model drift	Baselines adapt continuously via EMA	Requires periodic retraining on new data
Adversarial resistance	Attacker must manipulate physical properties of data	Models can be fooled by adversarial inputs
Day-one readiness	Works immediately from baseline	Requires training or tuning period before production accuracy
Computational cost	Lightweight arithmetic	GPU-intensive inference
Inspectability	Transparent box -- every intermediate value visible and auditable	Black box -- weights and activations are opaque

How Math Detects What AI/ML Cannot

The Evidence Engine deploys 26 mathematical detectors organized into five families:

- **Statistical** -- Z-Score, CUSUM, Mahalanobis Distance, EMA, Hurst Exponent, Destination Frequency
- **Information-Theoretic** -- Shannon Entropy, Entropy Rate, Compression Ratio, Kolmogorov Complexity (NCD)
- **Structural / Distributional** -- Benford's Law, Zipf's Law, Fuzzy Pattern Hashing, Cosine Similarity, Jensen-Shannon Divergence, Entity Diff
- **Temporal / Frequency-Domain** -- Cadence Detection, Spectral FFT, Wavelet Decomposition
- **Behavioral Inversion** -- Symmetric Traffic, Fixed Payload, Time-of-Day Inversion, Reader->Writer

Inversion, Talker Goes Quiet, Schedule Drift, Ghost Session

Each family attacks the problem from a mathematically independent angle. An adversary who defeats one family still faces four others.

1. Shannon Entropy -- Encryption Has a Shape

Encrypted data has near-maximum entropy (~8.0 bits/byte). Normal backup data does not. This is not a learned pattern -- it is an information-theoretic fact.

When ransomware encrypts files, the entropy of those files shifts measurably. No signature database needed. No training required. The math is the same whether the ransomware was written yesterday or ten years ago.

Detection: If backup data entropy spikes from a baseline of 5.2 to 7.9 bits/byte, something encrypted it.

Handling partial encryption: Some ransomware encrypts only file headers (first 4KB) to evade full-file entropy analysis. This is why entropy is one detector in a composite pipeline -- partial encryption still disrupts compression ratios, changes delta patterns, and alters behavioral cadence. No single detector is sufficient. The composite catches what any individual method might miss.

2. Z-Score -- Statistical Deviation is Universal

A z-score measures how many standard deviations a value is from the mean. The Evidence Engine applies z-scores to:

- **Bytes per flow** -- sudden volume changes
- **Packet count** -- traffic pattern shifts
- **Flow duration** -- connection timing changes
- **Backup change rate** -- data mutation velocity

At a threshold of 3.0 standard deviations, the false positive rate is 0.3%. This is not a tuned parameter from a training set -- it is a property of the normal distribution.

Detection: If tonight's backup change rate is 4.1 standard deviations above the 30-day baseline, something abnormal happened. The math doesn't care what caused it.

3. Cosine Similarity -- Behavioral Shape Comparison

The Evidence Engine represents each entity's behavior as a feature vector (bytes, packets, duration) and compares it against its baseline using cosine similarity -- the angle between two vectors in n-dimensional space.

- **1.0** = identical behavioral shape
- **0.0** = completely different behavior

This captures proportional changes that absolute thresholds miss. A server that normally sends 1GB in 1000 packets over 60 seconds looks different from one sending 10GB in 10000 packets over 600 seconds -- but the *shape* is identical (cosine similarity ≈ 1.0). A server that suddenly sends 1GB in 10 packets over 2 seconds has the same volume but a completely different shape.

Detection: Ransomware changes the shape of data flow. The math measures the angle change.

A note on AI/ML: Cosine similarity is used inside many AI/ML models. The difference is transparency. In an AI/ML pipeline, cosine similarity is one operation buried inside hidden layers -- the output is a probability with no visible reasoning. In the Evidence Engine, the cosine similarity score is the output: visible, auditable, and reproducible by any third party.

4. Cadence Detection -- Timing Has a Rhythm

Normal systems produce predictable timing patterns. Backups run on schedules. Logs arrive at regular intervals. The Evidence Engine tracks:

- **Average interval** between events (Welford's algorithm for streaming variance)
- **Burst detection** -- sudden clusters of activity
- **Silence detection** -- unexpected gaps

A ransomware attack disrupts timing cadence before it disrupts data content. The math catches the rhythm change.

Detection: If a backup system that writes every 300+/-15 seconds suddenly writes 47 times in 10 seconds, the cadence detector fires -- before any content analysis is needed.

5. Compression Ratio -- Redundancy is Measurable

Normal data compresses predictably. Encrypted data does not compress at all (it's already maximum entropy). The Evidence Engine tracks compression ratios via delta compression with XOR differencing.

- **Normal backup delta:** 3:1 to 10:1 compression ratio (redundant, compressible)
- **Encrypted backup delta:** ~1:1 compression ratio (random, incompressible)

Detection: When the compression ratio of backup deltas drops from 5:1 to 1.02:1, the data has been encrypted. No signatures needed.

6. Fuzzy Pattern Hashing -- Structure, Not Content

The Evidence Engine normalizes log entries by replacing variable content (IPs, timestamps, UUIDs, numbers) with tokens, then hashes the structural skeleton. Logs with 80%+ structural similarity cluster together automatically.

This creates behavioral fingerprints without AI/ML classification. When a new log type appears or existing patterns shift, the math detects it as a structural anomaly.

Detection: A server that normally produces 12 log pattern types suddenly producing 47 pattern types indicates compromise -- measured by clustering math, not AI/ML classification.

7. Exponential Moving Average -- Trend Without Training

Risk scores are smoothed using exponential moving average ($\alpha=0.1$): 10% weight on new observations, 90% on historical baseline. This eliminates single-event noise while catching sustained trends.

Unlike AI/ML models that need retraining when behavior shifts, EMA adapts continuously and transparently. The formula is auditable. The weights are visible. There is no hidden layer.

8. CUSUM -- Catching Slow Drift That Z-Scores Miss

Cumulative Sum (Page, 1954) is the mathematically optimal detector for small persistent shifts in a mean. Where a z-score asks "is this single observation extreme?", CUSUM asks "has the process been consistently drifting?"

The Evidence Engine runs two-sided CUSUM on bytes, packets, and duration per source. It accumulates deviations over time. A slow data exfiltration that adds 2% more bytes per flow -- invisible to z-scores -- will trigger CUSUM after enough samples accumulate.

Detection: An attacker slowly exfiltrating data at 2% above normal per flow goes unnoticed by z-score thresholds. CUSUM accumulates the deviation: after 50 flows, the cumulative sum crosses the alarm threshold. Math catches the drift.

9. Mahalanobis Distance -- Correlated Anomalies

Z-scores treat each metric independently. Mahalanobis distance accounts for correlations between them. A server running 80% CPU with 90% memory is normal (they correlate). The same server at 80% CPU with 10% memory is suspicious -- the correlation is broken.

The Evidence Engine tracks a full 3x3 covariance matrix (bytes, packets, duration) using Welford's online algorithm, inverts it via Cramer's rule, and computes $D^2 = (x-\mu)^T \Sigma^{-1} (x-\mu)$. No external math libraries. Pure Go arithmetic.

Detection: A compromised server producing high bytes but unusually low packets (large payloads, few connections -- data staging) scores normal on individual z-scores but anomalous on Mahalanobis because the metric correlation is broken.

10. Benford's Law -- Nature Has a Digit Distribution

The first digits of naturally occurring numbers follow $P(d) = \log_{10}(1 + 1/d)$. Digit 1 appears 30.1% of the time. Digit 9 appears 4.6%. This is not a learned rule -- it is a mathematical constant derived from scale invariance.

Packet sizes, byte counts, flow durations, and transaction amounts in normal enterprise traffic all follow Benford's Law. Synthetic data, fabricated logs, and attack-generated traffic violate it.

Detection: The Evidence Engine accumulates leading digits and runs a chi-squared goodness-of-fit test against the Benford distribution. If DGA-generated domain lookups produce uniformly distributed byte counts, the chi-squared statistic spikes. No training set. The expected distribution is a mathematical constant.

11. Zipf's Law -- Distribution Shape Has a Signature

In natural datasets, the k-th most frequent item appears with frequency proportional to $1/k^s$. Normal DNS queries, log categories, and API calls follow a steep Zipf distribution (exponent $s \approx 1.0-2.0$) -- a few items dominate, most are rare.

Attacks flatten this distribution. DGA domains, brute-force login attempts, and network reconnaissance spread activity across many targets equally, dropping the Zipf exponent below 0.5.

Detection: The Evidence Engine fits the Zipf exponent via log-log linear regression. An R^2 near 1.0 means the distribution is healthy. A flattened exponent with low R^2 means the natural power-law structure has been disrupted.

12. Spectral Analysis (FFT) -- Finding Hidden Frequencies

The Evidence Engine runs a full Cooley-Tukey radix-2 FFT on time-series data, transforming from time domain to frequency domain. Normal enterprise traffic has natural periodicities: 24-hour cycles, weekly patterns, maintenance windows. C2 beaconing introduces new, artificial frequencies.

Spectral flatness measures how concentrated energy is across frequencies. Low flatness (energy concentrated in few bins) with a dominant peak indicates periodic behavior -- a mathematical signature of C2 beaconing.

Detection: A compromised host beaconing to a C2 server every 60 seconds creates a sharp spectral peak at 1/60 Hz. No signature database needed. The FFT reveals the hidden rhythm. Pure math, implemented in Go without external libraries.

13. Entropy Rate -- Sequential Randomness, Not Just Static

Shannon entropy measures the randomness of a single distribution. Entropy rate measures the randomness of a p

rocess over time -- how predictable is the next state given the current one?

Normal protocols are structured state machines with low entropy rate (the next packet type is predictable from the current one). Encrypted tunnels, covert channels, and compromised processes have high entropy rate -- every transition is unpredictable.

Detection: The Evidence Engine builds a Markov transition matrix from observed state sequences, computes the stationary distribution, and calculates $h = -\sum p_i \log p_i$. A DNS-tunneling attack that encodes data in query strings produces near-maximum entropy rate. The math detects the hidden channel.

14. Wavelet Decomposition (Haar DWT) -- Anomalies at Every Timescale

Different anomalies manifest at different timescales. A packet storm is a seconds-scale event. A service disruption is minutes-scale. Gradual capacity exhaustion is hours-scale. A single detection window catches only one scale.

The Evidence Engine performs Haar wavelet decomposition, splitting signals into multiple resolution levels simultaneously. Energy analysis at each scale reveals where anomalies live. Donoho's universal threshold ($T = \sigma \sqrt{2 \log n}$) -- derived from MAD-based robust noise estimation -- separates signal from noise without any training.

Detection: A DDoS ramp-up shows low energy at fine scales (individual packets look normal) but high energy at coarse scales (the aggregate trend is anomalous). Wavelet decomposition catches it at the right timescale. Z-scores and entropy, operating at a single scale, miss it.

15. Kolmogorov Complexity (NCD) -- Algorithmic Incompressibility

Shannon entropy measures statistical randomness. Kolmogorov complexity measures algorithmic randomness -- the length of the shortest program that can reproduce the data. Encrypted, obfuscated, or AI-generated data has near-maximal complexity (incompressible). Normal operational data has lower complexity (repeating structure, compressible).

The Evidence Engine approximates Kolmogorov complexity via Normalized Compression Distance (NCD) using DEFLATE. It tracks the normalized complexity of data chunks over time and computes NCD between the current window and the historical baseline: $NCD(x,y) = (C(xy) - \min(C(x), C(y))) / \max(C(x), C(y))$, where $C()$ is compressed size.

Detection: A backup volume that normally has complexity 0.35 (structured, compressible) suddenly shows complexity 0.97 (incompressible). The NCD between current and baseline spikes to 0.9. The data has been encrypted or replaced with synthetic content -- detected without signatures, without training, using only the mathematical properties of compression.

16. Jensen-Shannon Divergence -- Distribution Drift Detection

Benford's Law and Zipf's Law test against specific theoretical distributions. Jensen-Shannon Divergence (JSD) generalizes this to compare *any* two empirical distributions -- packet size histograms, byte value distributions, inter-arrival time profiles, or API call frequency spectra.

JSD is symmetric and bounded: $JSD(P||Q) = 0.5 KL(P||M) + 0.5 KL(Q||M)$, where $M = 0.5(P+Q)$ is the mixture distribution. Range: 0.0 (identical) to 1.0 (maximally different). Unlike raw KL divergence, JSD is always finite and never requires both distributions to share the same support.

Detection: An attacker subtly shifts the packet size distribution -- slightly larger payloads, slightly different frequency profile -- to evade threshold-based detectors. JSD catches the distribution drift even when no individual value crosses any threshold. The math measures the shape of the entire distribution, not just its outliers.

17. Hurst Exponent -- Long-Range Memory Disruption

The Hurst exponent (H) measures the long-term memory of a time series via Rescaled Range (R/S) analysis. Normal system behavior exhibits persistent long-range dependence ($H > 0.5$) -- today's traffic correlates with yesterday's. Random noise has $H \sim 0.5$. Anti-persistent (choppy, erratic) behavior has $H < 0.5$.

The Evidence Engine computes H over sliding windows using R/S analysis at multiple segment sizes, fits the exponent via log-log regression, and tracks shifts in H over time. A sudden drop in H indicates that the system's natural temporal "memory" has been disrupted.

Detection: A file server that normally shows persistent write patterns ($H \sim 0.72$) suddenly drops to $H \sim 0.41$ during a ransomware encryption event -- the natural long-range correlation of backup writes is replaced by choppy, anti-persistent noise. The Hurst exponent catches the moment the system's rhythm breaks, even before volume or entropy change.

18. Destination Frequency -- Connection Rarity Scoring

The Evidence Engine tracks how frequently each entity communicates with each destination. A destination seen thousands of times is expected. A destination never seen before -- or seen only once in the past 30 days -- is rare and potentially suspicious.

Unlike blocklists or reputation databases, destination frequency requires no external threat intelligence. It measures rarity from the entity's own behavioral history. A new C2 server, a staging host used for the first time, or a cloud service never previously accessed all produce low frequency scores.

Detection: A workstation that normally connects to 15 well-known destinations suddenly connects to an IP it has never contacted before. The destination frequency score drops to 0.0. Combined with other detector signals, this identifies first-contact communications characteristic of initial C2 establishment.

19. Entity Diff -- Field-Level Change Detection

The Evidence Engine compares sequential evidence blobs for the same entity, performing field-by-field analysis to detect structural changes in the data. Each change is classified by type (addition, modification, deletion) and severity (low, medium, high, critical).

This goes beyond simple "did the data change?" to answer "what exactly changed, how severe is the change, and is this change consistent with the entity's history?" A new field appearing in a log entry, a permission field changing from "read" to "admin," or a configuration value being deleted are all detected and classified.

Detection: A database server's evidence blobs show a new "GRANT ALL PRIVILEGES" field that never existed before -- classified as a critical addition. A firewall rule entity shows a port range widened from "443" to "1-65535" -- classified as a critical modification. The math is structural comparison, not pattern matching.

20. Symmetric Traffic -- Tunneling Detection via Ratio Analysis

Normal client-server communication is asymmetric. A web browser sends small requests and receives large responses ($\text{bytesOut} \ll \text{bytesIn}$). An upload sends large payloads with small acknowledgments ($\text{bytesOut} \gg \text{bytesIn}$). The ratio of bytesOut to bytesIn reveals the direction and nature of communication.

When an attacker establishes a tunnel -- bidirectional data multiplexed over a protocol designed for one-directional content -- the traffic becomes symmetric. The ratio approaches 1.0.

Detection: An entity with a historical $\text{bytesOut}/\text{bytesIn}$ ratio of 0.05 (normal web browsing) suddenly shows a ratio of 0.85. The traffic has become symmetric -- a mathematical signature of tunneling, reverse shells, or covert channel activity. No deep packet inspection needed. The ratio is the signal.

21. Fixed Payload -- C2 Block Size Detection

Command-and-control systems that use fixed-size encrypted blocks produce response payloads with near-zero size variation. Normal HTTP responses vary widely -- HTML pages, images, API responses all differ in size. C2 responses using AES-CBC or similar block ciphers produce payloads in fixed multiples.

The Evidence Engine computes the coefficient of variation ($CV = \text{standard_deviation} / \text{mean}$) of response payload sizes per entity. Normal traffic: $CV > 0.3$. C2 with fixed blocks: $CV < 0.01$.

Detection: A host receiving responses where every payload is exactly 4,096 bytes ($CV \approx 0.0$) while all other hosts show $CV > 0.5$. The fixed-size pattern is a mathematical signature of encrypted block-cipher C2 communications -- invisible to content inspection but obvious to variance analysis.

22. Time-of-Day Inversion -- After-Hours Compromise Detection

The Evidence Engine maintains a 24-hour activity histogram for each entity. When activity occurs during hours that account for less than 2% of the entity's historical traffic, it is flagged as a temporal inversion.

This detects account takeover from different time zones, after-hours automated attacks, and compromised service accounts being used outside their normal operational windows -- without requiring any predefined "business hours" configuration.

Detection: A user account that has generated 99% of its traffic between 8am and 6pm EST for the past 90 days suddenly shows activity at 3am. The hour-3 bin contains 0.1% of historical activity. The math flags the temporal inversion -- the *when* is as important as the *what*.

23. Reader->Writer Inversion -- Role Change Detection

The Evidence Engine tracks the ratio of read operations to write operations for each entity. Most entities have a stable read-write profile. Monitoring accounts read but never write. Web servers read configs and write logs. Database replicas read but don't write.

When a historically read-heavy entity begins producing significant write operations, the role has inverted -- a mathematical signal of compromise.

Detection: A monitoring service account with a historical write ratio of 0.02 (98% reads, 2% writes) suddenly shows a write ratio of 0.65. The entity's role has inverted -- it is now writing more than reading. This indicates the account has been compromised and is being used for data modification, exfiltration staging, or lateral movement. The math is a single ratio comparison.

24. Talker Goes Quiet -- Communication Silence Detection

The Evidence Engine tracks the mean inter-message interval for entities that regularly send communications. When silence exceeds twice the mean interval, the entity is flagged.

This detects Business Email Compromise (BEC) scenarios where an attacker, having gained access to an executive's email account, suppresses outgoing messages to prevent the legitimate user from seeing fraudulent wire transfer requests or other attacker-initiated communications.

Detection: An executive's email account that sends messages every 47 minutes on average goes silent for 3 hours. The silence exceeds 2x the baseline interval. The math detects the absence of expected communication -- the dog that didn't bark.

25. Schedule Drift -- Cadence Deviation for Scheduled Tasks

Automated systems run on schedules: backups every 6 hours, log rotations every 24 hours, health checks every 5 minutes. The Evidence Engine computes $\text{drift} = |\text{actual_interval} - \text{expected_interval}| / \text{expected_interval}$, with 10% jitter tolerance.

When drift exceeds 50%, the scheduled task is no longer executing at its expected cadence -- indicating task replacement, clock manipulation, or system compromise that has disrupted the automation.

Detection: A backup job that runs every 360 minutes (+/-36 minutes jitter) suddenly runs at 847 minutes. $\text{Drift} = |847 - 360| / 360 = 135\%$. The schedule has been disrupted. No cron parsing or config file reading needed -- the math measures actual execution timing against its own historical cadence.

26. Ghost Session -- Login Without Logout Detection

The Evidence Engine identifies sessions where a login event occurred but no subsequent activity and no logout event followed. Sessions age against a per-entity baseline. As session age exceeds the entity's normal session duration, the anomaly score degrades.

Ghost sessions indicate credential testing (login, confirm access, disappear), session token theft (stolen token used once to validate, then held for later), and automated account enumeration (mass login attempts with no follow-up).

Detection: A login event from an IP that has never previously accessed the system, followed by zero activity for 45 minutes, with no logout. The entity's baseline session duration is 12 minutes. The session is 3.75x baseline with no activity -- a ghost. The math requires no threat intelligence. The absence of expected behavior is the signal.

Why This Matters

The Evidence Engine is a full-spectrum security detection platform that applies 26 mathematical detectors across every dimension of enterprise infrastructure. The 223 detection tiles span 16 categories covering the complete threat landscape:

Network Security

- Lateral movement detection via cosine similarity on behavioral vectors
- C2 beaconing identification via FFT spectral analysis
- DNS tunneling detection via entropy rate on query payloads
- Covert channel discovery via NCD on traffic patterns
- Non-standard port and protocol abuse via destination frequency analysis

Cloud Infrastructure

- Config drift detection via entity diff on cloud resource state
- Resource hijacking identification via Z-Score on compute metrics
- Unauthorized API detection via destination frequency + JSD
- Cloud image implant detection via NCD on image layers
- Unused cloud region activity via destination frequency

Identity & Access

- Credential attacks (brute force, stuffing, spraying) via Z-Score + cadence
- Privilege escalation via entity diff + CUSUM
- Ghost session detection -- login without logout, a behavioral inversion no other platform can detect
- MFA bypass and fatigue attack detection via cadence analysis
- Kerberos ticket forgery detection via JSD on ticket patterns

Application Security

- SQL injection, XSS, path traversal via entropy rate + NCD on request patterns
- API abuse via Z-Score + CUSUM + cadence
- Content injection and defacement via NCD on response baselines

Email & Phishing

- Business Email Compromise via entity diff + JSD
- Email silence detection -- Talker Goes Quiet, a behavioral inversion only the EE can measure
- Internal spearphishing via entropy rate on email bodies
- Spearphishing via third-party services

Endpoint Security

- Ransomware detection via entropy + compression ratio + CUSUM
- Anomalous process execution via entity diff + Z-Score
- Rootkit detection via entity diff + Shannon entropy on kernel modules
- Process injection detection via entity diff + Z-Score + NCD

Backup & Compliance

- Backup integrity verification via hash chain + entity diff
- Schedule anomaly detection via cadence + spectral/FFT
- Retention violation tracking via CUSUM + cadence
- Air-gap breach detection via cosine similarity

Data Integrity & Evidence

- Cryptographic evidence chain -- tamper-evident SHA-256 hash-linked blob sequence
- Hash mismatch detection on stored evidence
- Timestamp manipulation detection via Benford's Law + cadence
- Log deletion detection via CUSUM + entity diff

Behavioral / Zero-Day Detection

- Unknown pattern detection via Mahalanobis + JSD
- Baseline deviation via cosine similarity
- Cadence shift detection via Hurst exponent
- Six Beyond MITRE exclusive detections (see next section)

Failure mode for attackers: An adversary would need to evade all 26 mathematical detectors -- drawn from five

orthogonal families -- simultaneously, across every affected dimension. Evading one detector is plausible. Evading 26 independent detectors that measure fundamentally different physical and statistical properties of data simultaneously is computationally and operationally infeasible.

Beyond MITRE ATT&CK: EE-Exclusive Detections

The MITRE ATT&CK framework maps known adversary tactics, techniques, and procedures. It is the industry's most comprehensive catalog of attacker behavior. But MITRE maps what attackers **do** -- it does not map the mathematical consequences of their actions on the physical properties of data.

The Evidence Engine includes six detection capabilities that have no MITRE equivalent -- behavioral and mathematical measurements that no signature, rule, or framework can express:

1. Behavioral Regime Change (Hurst Exponent)

Detects when an entity's time-series behavior transitions from mean-reverting ($H < 0.5$) to trending ($H > 0.5$) or vice versa. This indicates a fundamental shift in the process generating the data -- not a spike, not an anomaly, but a regime change. MITRE has no technique for "the system's statistical memory structure changed."

2. Multi-Scale Time Anomaly (Wavelet Decomposition)

Detects anomalies that manifest simultaneously across multiple time scales -- seconds, minutes, hours, and days. A sophisticated attack that looks normal at any single time window is revealed when wavelet decomposition shows correlated energy anomalies across scales. MITRE maps individual techniques, not multi-scale temporal correlations.

3. Distribution Shape Shift (Jensen-Shannon Divergence)

Detects subtle changes in the shape of behavioral distributions that don't trigger any threshold. The entire distribution drifts -- heavier tails, flatter peaks, shifted modes -- while every individual value remains within normal bounds. MITRE maps discrete events, not distribution morphology.

4. Algorithmic Complexity Shift (NCD)

Detects when data becomes more or less algorithmically complex -- more compressible (structured/encoded) or less compressible (encrypted/randomized) -- compared to its historical baseline. This measures the Kolmogorov complexity of the data itself. MITRE maps what tools do to data, not the mathematical complexity change of the data.

5. Composite Detector Consensus

Aggregates weak signals from all 26 detectors that individually fall below threshold. When multiple orthogonal detectors each register marginal deviations, their consensus produces high-confidence detection of novel attacks that no single detector would catch alone. MITRE has no concept of cross-family mathematical consensus.

6. Cryptographic Evidence Chain Integrity

Monitors the integrity of the evidence chain itself -- detecting attempts to tamper with, fork, or replay the hash-linked blob sequence. This is meta-detection: the system detects attacks on its own detection infrastructure.

MITRE maps attacks on systems, not attacks on mathematical proof chains.

Additionally, the Evidence Engine's behavioral inversion detectors (Ghost Session, Talker Goes Quiet, Role Inversion, Time-of-Day Inversion, Fixed Payload) detect compromise through the **absence or inversion** of expected behavior -- a category of detection that MITRE's technique-based taxonomy cannot express. MITRE maps what attackers do. The EE also maps what legitimate entities **stop doing**.

The Evidence Engine doesn't just match MITRE ATT&CK -- it surpasses it.

Why Attackers Cannot Evade Math

A natural question: if the detectors are transparent and their formulas are published, can an attacker reverse-engineer evasion? The answer is no, and the reason is mathematical: the constraints imposed by the five detector families are contradictory. Satisfying one creates exposure to another. There is no single evasion strategy that defeats all five families simultaneously.

Evading Entropy Requires Weakening Encryption

Encrypted data has entropy near 8.0 bits/byte. To evade entropy detection, an attacker must produce ciphertext with entropy below the detection threshold -- say, 6.0 bits/byte. But any encryption scheme that produces low-entropy output is a weak cipher. Low entropy means the output has statistical structure that can be exploited to recover the plaintext. The attacker faces a binary choice: strong encryption that the entropy detector catches, or weak encryption that cryptanalysis breaks. There is no middle ground -- entropy is an information-theoretic property of the ciphertext, not a tunable parameter.

Evading Compression Ratio Triggers Other Detectors

Encrypted data does not compress (ratio near 1:1). To evade compression ratio detection, the attacker must add redundant padding so deltas compress to 3:1 or better. But adding redundancy means the data is no longer maximum-entropy -- the NCD detector catches the complexity drop, and the Shannon entropy detector sees the entropy decrease. The evasion for one detector directly triggers two others. The attacker cannot simultaneously produce data that appears compressible (to evade compression ratio) and random (to maintain encryption strength).

Evading Cadence Rate-Limits the Attack

To avoid cadence detection, the attacker must write data at the same rate and rhythm as legitimate processes. A backup system that writes every 300 seconds means the attacker can only encrypt one batch of data every 300 seconds. At legitimate backup throughput, encrypting a 10TB dataset takes the same time as a normal backup cycle -- days, not hours. The attacker has been rate-limited to the speed of normal operations. Combined with compression and entropy constraints, the attacker is now producing small amounts of weakly encrypted data at slow speeds -- a mathematically neutered attack.

Evading Mahalanobis Constrains Exfiltration Volume

Mahalanobis distance detects anomalies in the joint distribution of bytes, packets, and duration. To evade it, the attacker must maintain the historical correlation between all three metrics. If the baseline shows that 1GB requires 10,000 packets over 600 seconds, then exfiltrating 10GB requires 100,000 packets over 6,000 seconds -- exactly the right proportions. But 100,000 packets over 6,000 seconds is 16.7 packets/second, which is normal

throughput. The attacker cannot exfiltrate faster than normal without breaking the correlation. Combined with cadence constraints, the attacker is now operating at exactly the speed and volume of legitimate traffic -- which means minimal actual damage per unit time.

Evading Zipf Reduces Attack Coverage

Scanning, brute-force, and DGA attacks spread activity across many targets uniformly. Zipf detection catches this flattened distribution. To evade it, the attacker must concentrate most activity on a few targets, following the natural power law where a few items dominate and most are rare. But concentrating on few targets means scanning fewer hosts, trying fewer passwords, or resolving fewer domains -- drastically reducing attack effectiveness. The attacker has traded attack coverage for evasion.

The Mathematical Trap

Each evasion strategy satisfies one detector family at the cost of exposing the attacker to another. The five families form a closed mathematical trap: an attacker who satisfies all five constraints simultaneously is operating at the speed, volume, timing, correlation structure, and frequency distribution of legitimate traffic -- which means the attack has been reduced to a rate indistinguishable from normal operations. The math does not need to be secret. It needs to be correct.

Distinguishing Attacks from Legitimate Changes

A common question: how does math-based detection distinguish ransomware encryption from an administrator enabling BitLocker, or a user compressing files?

The answer is **context correlation across multiple detectors**:

- **Legitimate encryption** (BitLocker rollout) is preceded by change tickets, happens during maintenance windows, follows cadence patterns, and is performed by authorized processes. The cadence detector sees scheduled activity. The z-score sees expected volume changes.
- **Ransomware encryption** happens outside maintenance windows, disrupts timing cadence, produces abnormal process patterns, and mutates data at rates that deviate from baseline. Multiple detectors fire simultaneously.

A single entropy spike is not an alert. A simultaneous entropy spike + cadence disruption + compression ratio collapse + z-score deviation across multiple metrics is. The composite pipeline requires convergence across independent mathematical dimensions before flagging an anomaly.

Addressing false positives: Alert fatigue is the silent killer of security operations. AI/ML classifiers tune a single confidence threshold and accept the resulting false-positive rate. The Evidence Engine uses the composite as a *consensus engine* -- an alert fires only when multiple independent detectors agree. Because each detector measures a different physical property of the data, correlated false positives across all of them are statistically rare. The result: high sensitivity to real attacks with a structurally low false-positive rate, without requiring threshold tuning on labeled attack data.

The Evidence Gap

Detection is only half the problem. After an attack, organizations need to **prove** their recovery is clean. This is

where AI/ML completely fails:

Requirement	AI/ML	Math
Reproducibility	Difficult to reproduce -- model versions, weights, and inference paths vary	Run the formula again -- identical result every time
Audit trail	Model weights are opaque	Hash chain is independently verifiable
Court testimony	"The model said 87% confident" -- challenged under Daubert for explainability	"Entropy was 7.94 bits/byte, 4.2sigma above baseline" -- reproducible, verifiable
Insurance proof	Self-attestation of tool output	Cryptographic proof with tamper-evident chain
Regulatory compliance	Vendor-dependent certification	NIST SP 800-53 CP-9 specifies cryptographic hashes
Third-party verification	Requires same model, same version, same weights	Anyone with SHA-256 can verify

44% of cyber insurance claims were denied in 2024. The leading cause: inability to prove controls were operational during the incident window. AI/ML tools produce outputs. Math produces evidence.

Composite Detection Pipeline

The Evidence Engine doesn't rely on any single method. It fuses all mathematical detectors using weighted composite scoring:

Real-Time Per-Record Detectors

Detector	Weight	What It Measures
Cosine Similarity	25%	Behavioral shape deviation
Mahalanobis Distance	20%	Correlated multivariate anomalies
Z-Score	15%	Statistical outlier magnitude
CUSUM	15%	Slow persistent drift detection
Cadence	15%	Timing rhythm disruption
Destination Frequency	10%	Connection rarity

Each detector produces a score from 0.0 (anomalous) to 1.0 (normal). The weighted composite provides a single, explainable risk score that combines 6 independent mathematical perspectives on the same data -- in real time, per record.

Windowed Batch Analyzers

Analyzer	What It Measures
Benford's Law	Digit distribution anomalies (fraud, synthetic data)
Zipf's Law	Frequency distribution flattening (DGA, brute-force)

Analyzer	What It Measures
Spectral / FFT	Hidden periodicities and C2 beaconing
Entropy Rate	Sequential randomness (covert channels, tunneling)
Wavelet DWT	Multi-timescale anomalies (ramp-up attacks, slow degradation)
Kolmogorov / NCD	Algorithmic incompressibility (encryption, synthetic/AI-generated data)
Jensen-Shannon Divergence	Arbitrary distribution drift (subtle payload changes)
Hurst Exponent	Long-range temporal memory disruption (behavioral regime change)

Behavioral Inversion Detectors

Detector	What It Measures
Entity Diff	Field-level changes between sequential evidence blobs (permissions, configs, structure)
Symmetric Traffic	Traffic symmetry ratio -- tunneling and covert channels
Fixed Payload	Coefficient of variation of response sizes -- encrypted C2 block detection
Time-of-Day Inversion	Activity during historically silent hours -- after-hours account takeover
Reader->Writer Inversion	Read-write ratio shift -- compromised read-only accounts
Talker Goes Quiet	Communication silence -- BEC mail suppression
Schedule Drift	Scheduled task cadence deviation -- task replacement, clock manipulation
Ghost Session	Login without logout -- credential testing, session theft, account enumeration

Behavioral inversion detectors identify compromise by detecting when a quantifiable behavioral property *inverts* from its established norm. They operate on the hash-chained evidence stream without endpoint agents, training data, or predefined rules. Each measures a different dimension of "normal" -- so an attacker who maintains one dimension still exposes another.

Windowed analyzers operate over batches of records. They detect structural patterns that no single record can reveal -- frequency domain signatures, distribution shapes, multi-scale energy concentrations, and long-range temporal correlations.

Why these weights: Cosine similarity (25%) captures the broadest behavioral shape change. Mahalanobis (20%) catches correlated anomalies that independent z-scores miss. Z-score (15%) and CUSUM (15%) provide complementary statistical views -- z-score for single-point outliers, CUSUM for accumulated drift. Cadence (15%) detects timing disruptions that precede data corruption. Destination frequency (10%) supplements with connection rarity. The eight batch analyzers provide independent structural validation. The eight behavioral inversion detectors add a third layer -- detecting not what the data looks like, but how the entity *behaves*. Critically, detectors span five orthogonal families (statistical, information-theoretic, structural, temporal, behavioral) so that no single evasion technique can suppress them all. Weights are configurable per deployment and can be tuned to specific environments without retraining a model.

Performance: Every detector is deterministic arithmetic -- addition, multiplication, division, logarithms. No GPU required. No model inference latency. The full 6-detector composite runs at line speed on commodity hardware; windowed batch analyzers process on configurable intervals. Total overhead is a fraction of what a single AI/ML inference pass costs.

No black box. No hidden layers. No training data. **A transparent box: 26 mathematical detectors, zero AI/ML dependencies, every intermediate value inspectable.**

Full MITRE ATT&CK Coverage

The 26 mathematical detectors map to all 14 MITRE ATT&CK tactics and 635 unique technique IDs -- covering the entire adversary lifecycle from initial reconnaissance through impact, plus six detections beyond the MITRE framework that no other platform offers.

MITRE Tactic	Technique IDs	Key Detectors
Reconnaissance (TA0043)	T1595, T1593, T1590, T1592, T1589, T1591, T1598, T1596, T1583-T1588, T1608 (15)	Destination Frequency, Entropy Rate, JSD, NCD, Cadence
Resource Development (TA0042)	T1583 Acquire Infrastructure, T1584 Compromise, T1587 Develop, T1588 Obtain, T1585/T1586 Accounts	Entropy Rate, NCD, Entity Diff, JSD
Initial Access (TA0001)	T1190 Exploit Public-Facing App, T1566 Phishing, T1189 Drive-by, T1199 Trusted Relationship, T1195 Supply Chain	Entropy Rate, NCD, JSD, Cadence, Entity Diff
Execution (TA0002)	T1047 WMI, T1053 Scheduled Task, T1059 Scripting, T1106 Native API, T1610 Container, T1648 Serverless, T1559 IPC, T1569 System Services, T1203 Client Exploit, T1129 Shared Modules (10 tiles)	Entity Diff, Z-Score, NCD, Cadence, Schedule Drift
Persistence (TA0003)	T1547 Autostart, T1037 Logon Scripts, T1176 Browser Ext, T1543 System Process, T1546 Event Triggered, T1133 External Remote, T1574 Hijack Flow, T1525 Container Implant, T1137 Office Startup, T1542 Pre-OS Boot, T1505 Server Component, T1205 Traffic Signal, T1078 Valid Accounts, T1098 Account Manip,	Entity Diff, Schedule Drift, Cadence, NCD, Destination Frequency
Privilege Escalation (TA0004)	T1134 Access Token Manip, T1548 Elevation Control, T1068 Exploit for Priv Esc	Entity Diff, CUSUM, Z-Score
Defense Evasion (TA0005)	T1197 BITS, T1140 Deobfuscation, T1611 Container Escape, T1006 Direct Volume, T1484 Domain Policy, T1222 File Permissions, T1564 Hidden Artifacts, T1070 Indicator Removal, T1562 Impair Defenses, T1202 Indirect Command, T1036 Masquerading, T1578 Cloud Infra Mod, T1112 Registry, T1599 Network Boundary, T1027 Obfuscated Files, T1055 Process Injection, T1620 Reflective Code, T1207 Rogue DC, T1014 Rootkit, T1553 Subvert Trust, T1218 System Binary Proxy, T1221 Template Injection, T1497 Sandbox Evasion (24 tiles)	CUSUM, Entity Diff, NCD, Entropy Rate, Shannon Entropy, JSD

MITRE Tactic	Technique IDs	Key Detectors
Credential Access (TA0006)	T1110 Brute Force, T1003 OS Credential Dumping, T1056 Input Capture, T1187 Forced Auth, T1606 Forge Web Creds, T1558 Kerberos, T1621 MFA Fatigue, T1649 Certificate Theft, T1539 Web Session, T1552 Unsecured Creds, T1531 Account Access	Z-Score, Cadence, Entropy Rate, JSD, Entity Diff
Discovery (TA0007)	Remote Access (20 tiles) T1007 A (20 tiles), T1580 Cloud Infra, T1069 Permission Groups, T1135 Network Share, T1057 Process, T1082 System Info, T1016 Network Config, T1049 Network Connections, T1518 Software, T1482 Domain Trust, T1033 System Owner, T1007 System Service, T1046 Network Scanning	Destination Frequency, Z-Score, Entity Diff, Cadence
Lateral Movement (TA0008)	T1021 Remote Services, T1210 Exploitation, T1563 Session Hijacking, T1072 Software Deployment, T1080 Taint Shared Content, T1570 Lateral Tool Transfer (8 tiles)	Cosine Similarity, JSD, Destination Frequency, NCD, Entity Diff
Collection (TA0009)	T1119 Automated Collection, T1114 Email Collection, T1530 Cloud Storage, T1113 Screen Capture, T1123 Audio, T1125 Video, T1115 Clipboard (8 tiles)	CUSUM, Entity Diff, Talker Goes Quiet, Z-Score
Command & Control (TA0011)	T1071 App Layer Protocol, T1573 Encrypted Channel, T1572 Tunneling, T1132 Data Encoding, T1001 Data Obfuscation, T1568 Dynamic Resolution, T1008 Fallback, T1104 Multi-Stage, T1095 Non-App Layer, T1571 Non-Standard Port, T1090 Proxy/Domain Fronting, T1219 Remote Access (16 tiles)	Cadence, FFT, Symmetric Traffic, Fixed Payload, NCD, Entropy Rate, JSD
Exfiltration (TA0010)	T1041 Over C2 Channel, T1048 Alt Protocol, T1020 Automated, T1567 Web Service, T1052 Physical Medium, T1030 Size Limits, T1074 Data Staged, T1560 Archive, T1039 Network Share, T1537 Cloud Account, T1557	Cosine Similarity, Z-Score, CUSUM, NCD, Compression Ratio, Symmetric Traffic
Impact (TA0040)	T1486 Data Encrypted, T1489 Service Stop, T1498 Network DoS, T1499 Endpoint DoS, T1561 Disk Wipe, T1495 Firmware, T1491 Defacement, T1659 Content Injection, T1485 Data Destruction, T1565 Data Manipulation	Shannon Entropy, Haar Wavelet, Z-Score, CUSUM, NCD, Entity Diff

(13 tiles)

AI/ML security tools typically cover 3-5 MITRE tactics and rely on threat intelligence feeds for the rest. The Evidence Engine covers all 14 tactics using mathematical detectors alone -- no signatures, no feeds, no models. Each detector operates on the physical and statistical properties of data, making coverage intrinsic rather than database-dependent.

The 223 detection tiles on the Evidence Engine dashboard map 635 MITRE technique IDs across all 14 tactics plus 6 Beyond MITRE exclusives, with each tile combining 1-3 mathematical detectors applied to a specific attack scenario. Complete MITRE ATT&CK coverage -- and beyond. Zero AI/ML dependencies.

Summary

AI/ML detects what it has seen before. Math detects what is happening now.

AI/ML tools are powerful for pattern recognition in domains with stable distributions and abundant labeled data. Cybersecurity has neither. Threats evolve daily. Novel attacks are the ones that matter most. And when the attack is over, you need proof that stands up in court -- not a probability score from a model no one can inspect.

The Evidence Engine runs 26 mathematical detectors -- Shannon entropy, z-scores, cosine similarity, cadence analysis, compression ratios, fuzzy pattern hashing, exponential moving averages, CUSUM, Mahalanobis distance, Benford's Law, Zipf's Law, spectral FFT analysis, Markov entropy rate, Haar wavelet decomposition, Kolmogorov complexity via NCD, Jensen-Shannon divergence, Hurst exponent, destination frequency scoring, field-level entity diff, symmetric traffic detection, fixed payload analysis, time-of-day inversion, reader-writer role inversion, communication silence detection, schedule drift measurement, and ghost session identification -- all implemented in pure Go with zero AI/ML dependencies, chained into a tamper-evident cryptographic audit trail. Every measurement is deterministic. Every result is reproducible. Every proof is independently verifiable.

CyberSense needs 7,500 known ransomware signatures to detect 7,500 known threats. The Evidence Engine needs zero signatures to detect any threat that changes the shape of data -- including the 7,501st variant that hasn't been written yet. These 26 detectors power 223 specific threat detections across 16 security categories, mapping 635 MITRE ATT&CK technique IDs across all 14 adversary lifecycle tactics -- plus 6 Beyond MITRE detections that no other platform offers. Complete framework coverage -- and beyond. Zero AI/ML dependencies.

AI/ML predicts. Math proves. This is Evidence Based Detection.